

1. Responsibility after Delegation

When an agent writes code, runs tests, delegates analysis, and returns a final answer, several claims are bundled together: the requested requirement was understood; the implementation matches it; relevant regressions were considered; checks actually ran; and any remaining uncertainty is acceptable. Review separates those claims.

2. Read-Only Review

Codex supports review workflows that analyze changes without modifying the working tree. This is useful because the reviewer is not simultaneously optimizing for "finish the patch." Its objective can be adversarial: find actionable defects introduced by the change, with attention to correctness, performance, security, maintainability, and developer experience.

A review that writes fixes immediately can erase evidence of what it first found. Read-only first, fix second, preserves the diagnostic step.

3. Review the Intent, Not Only the Diff

A technically clean diff can solve the wrong problem. The reviewer needs the task contract or issue context, not merely code lines. Compare every material requirement with an observable implementation or test. Ask which requirement has no evidence.

For example, "add offline mode without changing startup latency" has two claims. A diff may implement caching beautifully while never measuring startup. Review should expose the unsupported half.

4. Severity and Actionability

Useful review is selective. A reviewer that reports twenty stylistic nits alongside one data-loss bug increases noise. Prioritize findings that can change whether the patch should ship. Separate blocking correctness risks from optional improvements.

A finding should usually contain:

- the affected location or behavior;
- the concrete failure or risk;
- why it follows from the current code;
- a reproducer, counterexample, or condition when practical.

5. The Self-Review Objection

Can the implementer simply review its own work? Often yes for low-risk changes. The objection is correlated error: the same mistaken assumption may shape both patch and review. A fresh read-only turn, separate subagent, or human reviewer can increase independence.

Independence matters most when the patch is complex, the specification is ambiguous, the tests were written by the implementer, or consequences are high.

6. Verification versus Review

Verification asks whether specified checks pass. Review asks whether the chosen specification, implementation, and checks missed something important. They overlap but are not substitutes.

A parser can pass every existing test while mishandling an untested edge case visible from the grammar. Review may find it. Conversely, a reviewer can approve a convincing diff without executing the failing input. Verification may find the defect.

7. Responsibility Statement

A final handback should not claim certainty beyond its evidence. State what was changed, what was checked, what review occurred, and what remains unknown. This is especially important when tools or permissions prevented a relevant check.

References: OpenAI, Code review in Codex; OpenAI, Codex security; OpenAI, Codex use cases.

8. Worked Review: Schema Migration

Suppose an implementation changes a database column from nullable to required and adds a migration that fills missing rows. The unit tests pass. A useful reviewer asks questions the implementer may have skipped: Can new writes create null between deploy phases? Does the migration lock a large table? Can the application version deployed before the migration read the new state? Is rollback possible? Which invariant was proven by the tests, and which is only assumed from code reading?

The review should not demand every imaginable safeguard. It should identify the failures plausibly introduced by this diff and rank them by consequence.

9. Review Closure

After findings are fixed, the review loop needs closure. Re-run checks affected by the fix, then either rerun the focused review or verify each finding directly. Do not restart an unlimited audit after every small edit unless new evidence justifies it. A review process without a stopping rule can consume more effort than the change itself while adding diminishing information.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 13: Review Responsibility

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>